```
"""
================================================
  JARVIS - AI Voice Assistant
  Built by: Kshitij Maske
  Tech: Python, SpeechRecognition, pyttsx3,
        Wikipedia, Tkinter, Webbrowser
================================================
"""

import tkinter as tk
from tkinter import scrolledtext
import threading
import speech_recognition as sr
import pyttsx3
import wikipedia
import webbrowser
import datetime
import random
import time

# ── VOICE ENGINE SETUP ──────────────────────────────────────────────

engine = pyttsx3.init()

def set_female_voice():
    voices = engine.getProperty('voices')
    # Try to find a female voice
    for voice in voices:
        if any(word in voice.name.lower() for word in ['female', 'zira', 'hazel', 'susan',
'helena', 'woman']):
            engine.setProperty('voice', voice.id)
            return
    # If no female voice found, use second voice if available
    if len(voices) > 1:
        engine.setProperty('voice', voices[1].id)

set_female_voice()
engine.setProperty('rate', 175)    # Speed
engine.setProperty('volume', 1.0)  # Volume

# ── JOKES LIST ──────────────────────────────────────────────────────

JOKES = [
    "Why do programmers prefer dark mode? Because light attracts bugs!",
    "Why did the programmer quit his job? Because he didn't get arrays!",
    "How many programmers does it take to change a light bulb? None, that's a hardware
problem!",
    "Why do Java developers wear glasses? Because they don't C sharp!",
```

```
    "A SQL query walks into a bar, walks up to two tables and asks... Can I join you?",
    "Why was the computer cold? It left its Windows open!",
    "What do you call a computer that sings? A Dell!",
    "Why did the developer go broke? Because he used up all his cache!",
]

# ── GREETINGS ──────────────────────────────────────────────────────────────

def get_greeting():
    hour = datetime.datetime.now().hour
    if 0 <= hour < 12:   return "Good Morning"
    if 12 <= hour < 17:  return "Good Afternoon"
    return "Good Evening"

# ── CORE FUNCTIONS ─────────────────────────────────────────────────────────

def speak(text, app=None):
    """Speak text and show in UI."""
    if app:
        app.add_log(f"🤖 Jarvis: {text}", color="#00d4ff")
    engine.say(text)
    engine.runAndWait()

def listen(app=None):
    """Listen to microphone and return text."""
    r = sr.Recognizer()
    r.energy_threshold = 300
    r.dynamic_energy_threshold = True

    with sr.Microphone() as source:
        if app:
            app.set_status("🎤 Listening...", "#00ff9d")
            app.add_log("🎤 Listening...", color="#8b949e")
        r.adjust_for_ambient_noise(source, duration=0.5)
        try:
            audio = r.listen(source, timeout=5, phrase_time_limit=8)
        except sr.WaitTimeoutError:
            if app: app.set_status("⏱ Timeout — Try again", "#f59e0b")
            return None

    if app: app.set_status("⚙️ Processing...", "#7c3aed")

    try:
        query = r.recognize_google(audio, language="en-IN")
        if app:
            app.add_log(f"👤 You: {query}", color="#e6edf3")
            app.set_status("✅ Got it!", "#00ff9d")
        return query.lower()
    except sr.UnknownValueError:
        speak("Sorry, I didn't catch that. Could you repeat?", app)
```

```
        if app: app.set_status("❓ Didn't understand", "#f59e0b")
        return None
    except sr.RequestError:
        speak("I'm having trouble connecting to the internet.", app)
        if app: app.set_status("❌ Network error", "#ef4444")
        return None

# ── COMMAND HANDLERS ──────────────────────────────────────────────────────────

def open_website(query, app):
    sites = {
        "youtube":   ("https://youtube.com",   "Opening YouTube for you!"),
        "google":    ("https://google.com",    "Opening Google!"),
        "github":    ("https://github.com",    "Opening GitHub!"),
        "instagram": ("https://instagram.com", "Opening Instagram!"),
        "whatsapp":  ("https://web.whatsapp.com", "Opening WhatsApp Web!"),
        "linkedin":  ("https://linkedin.com",  "Opening LinkedIn!"),
        "internshala": ("https://internshala.com", "Opening Internshala!"),
        "gmail":     ("https://mail.google.com", "Opening Gmail!"),
        "facebook":  ("https://facebook.com",  "Opening Facebook!"),
    }
    for site, (url, msg) in sites.items():
        if site in query:
            speak(msg, app)
            webbrowser.open(url)
            return True
    return False

def tell_time(app):
    now = datetime.datetime.now()
    time_str = now.strftime("%I:%M %p")
    speak(f"The current time is {time_str}", app)

def tell_date(app):
    now = datetime.datetime.now()
    date_str = now.strftime("%A, %d %B %Y")
    speak(f"Today is {date_str}", app)

def search_wikipedia(query, app):
    speak("Searching Wikipedia, please wait...", app)
    query = query.replace("wikipedia", "").replace("search", "").strip()
    try:
        wikipedia.set_lang("en")
        result = wikipedia.summary(query, sentences=2)
        speak(f"According to Wikipedia: {result}", app)
    except wikipedia.exceptions.DisambiguationError as e:
        speak(f"There are multiple results. Try being more specific.", app)
    except wikipedia.exceptions.PageError:
        speak(f"Sorry, I couldn't find anything about {query} on Wikipedia.", app)
    except Exception:
```

```
        speak("Sorry, something went wrong with the Wikipedia search.", app)

def tell_joke(app):
    joke = random.choice(JOKES)
    speak(joke, app)

def search_google(query, app):
    search_query = query.replace("search", "").replace("google", "").strip()
    speak(f"Searching Google for {search_query}", app)
    webbrowser.open(f"https://www.google.com/search?q={search_query}")

def process_command(query, app):
    """Match query to command and execute."""
    if not query:
        return

    # Greet back
    if any(w in query for w in ["hello", "hi", "hey", "jarvis"]):
        speak(f"Hello Kshitij! How can I help you?", app)

    # Time
    elif any(w in query for w in ["time", "what time"]):
        tell_time(app)

    # Date
    elif any(w in query for w in ["date", "today", "day"]):
        tell_date(app)

    # Wikipedia
    elif "wikipedia" in query or "who is" in query or "what is" in query:
        search_wikipedia(query, app)

    # Joke
    elif any(w in query for w in ["joke", "funny", "laugh", "jokes"]):
        tell_joke(app)

    # Open websites
    elif any(w in query for w in ["open", "launch", "go to", "play"]):
        if not open_website(query, app):
            speak("Sorry, I don't know that website yet.", app)

    # Google search
    elif "search" in query:
        search_google(query, app)

    # Who are you
    elif any(w in query for w in ["who are you", "your name", "introduce"]):
        speak("I am Jarvis, your personal AI voice assistant, built by Kshitij Maske!", app)

    # Thanks
```

```
    elif any(w in query for w in ["thank", "thanks", "thank you"]):
        speak("You're welcome, Kshitij! Always here to help.", app)

    # Stop / Exit
    elif any(w in query for w in ["stop", "exit", "bye", "goodbye", "quit"]):
        speak("Goodbye Kshitij! Have a great day!", app)
        if app: app.after(1500, app.destroy)

    # Help
    elif "help" in query or "commands" in query:
        speak("You can ask me to open YouTube, Google, GitHub, or any website. Ask me the time
or date. Search Wikipedia. Tell you a joke. Or search Google for anything!", app)

    else:
        speak(f"I'm not sure how to help with that yet. Try asking me to open a website, tell
the time, or search Wikipedia!", app)

# ── GUI ──────────────────────────────────────────────────────────────────────

class JarvisApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("JARVIS — Voice Assistant by Kshitij Maske")
        self.geometry("780x580")
        self.configure(bg="#080b10")
        self.resizable(False, False)

        self.is_listening = False
        self.build_ui()
        self.after(800, self.startup_greeting)

    def build_ui(self):
        # Header
        hdr = tk.Frame(self, bg="#0a0f16", pady=18)
        hdr.pack(fill="x")

        tk.Label(hdr, text="J.A.R.V.I.S", bg="#0a0f16",
            fg="#00d4ff", font=("Consolas", 26, "bold")).pack()
        tk.Label(hdr, text="Just A Rather Very Intelligent System",
            bg="#0a0f16", fg="#8b949e",
            font=("Consolas", 9)).pack()
        tk.Label(hdr, text="Built by Kshitij Maske",
            bg="#0a0f16", fg="#30363d",
            font=("Consolas", 8)).pack()

        # Status ring
        self.status_frame = tk.Frame(self, bg="#080b10", pady=12)
        self.status_frame.pack(fill="x")
        self.status_var = tk.StringVar(value="🟢 Ready — Click Listen or type a command")
        self.status_lbl = tk.Label(self.status_frame,
```

```
        textvariable=self.status_var,
        bg="#080b10", fg="#00ff9d",
        font=("Consolas", 10))
    self.status_lbl.pack()

    # Log area
    log_frame = tk.Frame(self, bg="#080b10", padx=20)
    log_frame.pack(fill="both", expand=True)

    self.log = scrolledtext.ScrolledText(log_frame,
        bg="#0d1117", fg="#e6edf3",
        font=("Consolas", 10),
        relief="flat", wrap="word",
        state="disabled", height=16,
        insertbackground="#00d4ff",
        highlightthickness=1,
        highlightbackground="#1e2a3a")
    self.log.pack(fill="both", expand=True)
    self.log.tag_config("jarvis", foreground="#00d4ff")
    self.log.tag_config("user",   foreground="#e6edf3")
    self.log.tag_config("muted",  foreground="#8b949e")
    self.log.tag_config("green",  foreground="#00ff9d")
    self.log.tag_config("red",    foreground="#ef4444")

    # Text input row
    input_frame = tk.Frame(self, bg="#080b10", padx=20, pady=10)
    input_frame.pack(fill="x")

    self.text_input = tk.Entry(input_frame,
        bg="#161b22", fg="#e6edf3",
        font=("Consolas", 11), relief="flat",
        insertbackground="#00d4ff",
        highlightthickness=1,
        highlightcolor="#00d4ff",
        highlightbackground="#30363d")
    self.text_input.pack(side="left", fill="x", expand=True, ipady=8, padx=(0,10))
    self.text_input.bind("<Return>", self.on_text_enter)

    tk.Button(input_frame, text="Send ➤",
        bg="#00d4ff", fg="#080b10",
        font=("Consolas", 10, "bold"),
        relief="flat", cursor="hand2",
        padx=16, pady=6,
        command=self.on_text_enter).pack(side="left")

    # Big listen button
    btn_frame = tk.Frame(self, bg="#080b10", pady=14)
    btn_frame.pack(fill="x")

    self.listen_btn = tk.Button(btn_frame,
```

```
            text="🎤  CLICK TO SPEAK",
            bg="#00ff9d", fg="#080b10",
            font=("Consolas", 13, "bold"),
            relief="flat", cursor="hand2",
            padx=40, pady=14,
            command=self.toggle_listen)
        self.listen_btn.pack()

        # Commands hint
        hint = tk.Frame(self, bg="#080b10", pady=6)
        hint.pack(fill="x")
        tk.Label(hint,
            text='💡 Try: "Open YouTube" · "What time is it?" · "Tell me a joke" · "Search
Wikipedia Python"',
            bg="#080b10", fg="#30363d",
            font=("Consolas", 8)).pack()

    def set_status(self, text, color="#00ff9d"):
        self.status_var.set(text)
        self.status_lbl.config(fg=color)

    def add_log(self, text, color="#e6edf3"):
        self.log.config(state="normal")
        tag = "jarvis" if "Jarvis" in text else "user" if "You" in text else "muted"
        if color == "#00ff9d": tag = "green"
        if color == "#ef4444": tag = "red"
        self.log.insert("end", text + "\n", tag)
        self.log.see("end")
        self.log.config(state="disabled")

    def startup_greeting(self):
        greeting = get_greeting()
        msg = f"{greeting}, Kshitij! I am Jarvis, your personal voice assistant. How can I help
you today?"
        threading.Thread(target=speak, args=(msg, self), daemon=True).start()
        self.set_status("🟢 Ready — Click Listen or type a command")

    def toggle_listen(self):
        if not self.is_listening:
            self.is_listening = True
            self.listen_btn.config(text="🔴  LISTENING...", bg="#ef4444")
            threading.Thread(target=self.listen_and_process, daemon=True).start()

    def listen_and_process(self):
        query = listen(self)
        self.is_listening = False
        self.listen_btn.config(text="🎤  CLICK TO SPEAK", bg="#00ff9d")
        self.set_status("🟢 Ready", "#00ff9d")
        if query:
            process_command(query, self)
```

```
    def on_text_enter(self, event=None):
        text = self.text_input.get().strip()
        if not text: return
        self.text_input.delete(0, "end")
        self.add_log(f"👤 You: {text}", color="#e6edf3")
        threading.Thread(target=process_command, args=(text.lower(), self), daemon=True).start()


# ── RUN ──────────────────────────────────────────────────────────────────────

if __name__ == "__main__":
    app = JarvisApp()
    app.mainloop()
```